

Reporte de Agentic Loop

0. SKILLS CONFIGURADOS

arquitecto !' DISEÑO-ARQUITECTURA (GLX-DISEÑO-ARQUITECTURA-V1)
tech_lead !' MICROPLANIFICACION-DISEÑO (GLX-MICROPLANIFICACION-DISEÑO-V1)
desarrollador !' FRONTEND-DESARROLLO (GLX-FRONTEND-DESARROLLO-V1)
qa !' test (GLX-QA-VALIDACION-V1)

1. ANÁLISIS DE ARQUITECTURA

Micro-ADR: XP-14 — Página Web Metodología TI

1. CONTEXTO

Se requiere una página web informativa sobre metodologías de informática y TI con 3 secciones principales (Home, Historias, Noticias), FAQ, aviso de cookies, Google Analytics y medidas básicas de seguridad XSS. El stack está explícitamente restringido a HTML5, CSS y JS puro. No hay backend, no hay autenticación, no hay base de datos. El entorno de entrega es un archivo estático que puede servirse desde un servidor HTTP simple o abrirse localmente.

2. DECISIÓN

****Arquitectura: Single-File Application (SFA) con archivos separados por responsabilidad, TODO el JS consolidado en un único archivo `main.js` plano (sin ES Modules / import-export).****

Se elige un sitio estático multi-sección con navegación SPA simulada mediante hash-routing en JS vanilla. Un solo `index.html` contiene las 3 secciones como divs ocultos/visibles. CSS en un único `styles.css`. JS en un único `main.js`.

3. JUSTIFICACIÓN (YAGNI + KISS)

- ****No se usa React/Vue/Angular****: YAGNI. El requerimiento es contenido estático informativo. Un framework JS añadiría 200KB+ de bundle, tiempo de compilación y complejidad de despliegue sin ningún beneficio funcional para este caso.
- ****No se usa bundler (Webpack/Vite)****: YAGNI. Sin módulos ES6 nativos y con un solo archivo JS, no hay nada que empaquetar. Elimina una dependencia de infraestructura completa.
- ****No se divide en múltiples archivos JS****: El entorno debe soportar apertura local (file:///) según criterios de aceptación del skill. Para garantizar compatibilidad máxima y evitar errores CORS con módulos ES6, todo el JS va en `main.js` como script clásico.
- ****No se usa CMS ni backend****: YAGNI. El contenido es estático y editorial. Un backend añadiría servidor, base de datos y superficie de ataque sin valor para el MVP.
- ****No se usa MongoDB, RabbitMQ ni servicios externos de datos****: El único servicio externo permitido es Google Analytics (requerimiento explícito) y DOMPurify (CDN, requerimiento de seguridad explícito).
- ****Hash-routing en lugar de History API****: Evita la necesidad de configuración de servidor para reescritura de URLs, manteniendo el sitio 100% estático y portable.

4. SEGURIDAD (Criterios explícitos del requerimiento)

- ****DOMPurify****: Cargado desde CDN con SRI hash para sanitizar cualquier input de usuario (buscador, comentarios FAQ).
- ****CSP Meta tag****: `` restringiendo `script-src` a `self` + dominios de Google Analytics y CDN de DOMPurify.
- ****Escape de caracteres****: Función `escapeHTML()` en `main.js` para cualquier interpolación dinámica de strings en el DOM.
- ****HTTPS****: Documentado en `README.md` como requisito de despliegue (Netlify/GitHub Pages lo proveen por defecto). No es implementable en archivos estáticos per se.
- ****Cookie Consent****: Banner propio en JS vanilla, almacena preferencia en `localStorage`. Bloquea la carga de Google Analytics hasta obtener consentimiento (patrón consent-first).

5. ACCESIBILIDAD (WCAG 2.1 AA)

- Uso de landmarks semánticos: ``<header>``, ``<nav>``, ``<main>``, ``<section>``, ``<footer>``, ``<article>``.
- `aria-label` en navegación y botones sin texto visible.
- Contraste de colores mínimo 4.5:1 definido en variables CSS.
- `skip-to-content` link como primer elemento del DOM.
- Focus visible en todos los elementos interactivos.

6. RIESGOS ASUMIDOS POR SIMPLIFICAR

- **Riesgo 1 (BAJO)**: Si el contenido crece a >50 artículos, el HTML inline se volverá difícil de mantener. Mitigación futura: migrar contenido a JSON estático y renderizar con JS.
- **Riesgo 2 (BAJO)**: Google Analytics requiere consentimiento GDPR. El banner de cookies implementado mitiga esto, pero no reemplaza una auditoría legal completa.
- **Riesgo 3 (MUY BAJO)**: Sin backend, el CSP se implementa via meta tag (no header HTTP), lo cual es menos robusto. Mitigación: documentar configuración de headers en servidor de despliegue en `README.md`.
- **Riesgo 4 (DESCARTADO)**: Microservicios, contenedores, SSR — completamente fuera de scope y explícitamente rechazados por YAGNI.

2. MÉTRICAS DE ITERACIÓN (QA)

- index.html | Estado: APROBADO | Iteraciones: 1
- assets/js/main.js | Estado: APROBADO | Iteraciones: 1
- assets/css/styles.css | Estado: APROBADO | Iteraciones: 1
- README.md | Estado: SKIPPED (doc) | Iteraciones: 0

3. MÉTRICAS DE TOKENS IA

Tokens de entrada (input): 123,533

Tokens de salida (output): 35,791

Total tokens: 159,324

Modelo utilizado: claude-sonnet-4-6

Ø=Üh Ø=Ü» ~15.4 horas-hombre ahorradas

"H squad de 4 personas × 2 días

Desglose por Agente:

DISEÑO-ARQUITECTURA: !"2,370 input | !"1,592 output | 1 llamadas | ~1.7h ahorradas

MICROPLANIFICACION-DISEÑO: !"19,352 input | !"9,500 output | 4 llamadas | ~6h ahorradas

FRONTEND-DESARROLLO: !"37,665 input | !"20,072 output | 4 llamadas | ~4.2h ahorradas

test: !"64,146 input | !"4,627 output | 4 llamadas | ~3.5h ahorradas